

TABLE IV: Comparison of object detection models on COCO [20].

Model	mAP	Parameters
RetinaNet [10]	41.5	38.2 millions
FCOS [8]	39.2	32.3 millions
Faster R-CNN [9]	37	41.8 millions
SSD [11]	25.1	35.6 millions

in Table IV. Among these models, RetinaNet [10] has the highest mAP, indicating superior performance in detecting and classifying objects within documents. We move forward with fine-tuning and testing the presented methods for stamp and signature detection.

Based on the comparisons made for keyword detection and stamp and signature detection, we summarize our choices as follows. We select Microsoft Azure OCR and Tesseract for OCR step, based on their high accuracy and similarity scores. Regarding document layout analysis, we move forward with LayoutLMv3, LayoutLMv2, and LiLT, to be mainly evaluated using the F1-score. It is worth noting that we consider standard models (not large ones, i.e., LayoutLMv3Large and LayoutLMv2Large) for their lower resource consumption and better fit for our data. For object detection, we benchmark RetinaNet, FCOS, Faster R-CNN, and SSD, using the mAP metric. These models are chosen based on their performance on benchmark datasets and their suitability for our specific tasks for invoice document validation.

III. RESULTS AND DISCUSSION

In this section, we present the results of the benchmarking performed on the selected models after fine-tuning them on our data. We also examine the challenge of class imbalance within the OCR-augmented dataset and its implications. In our experimental evaluation, we focus on precision, for a highly reliable invoice validation pipeline, with consideration of limited computational resources. The implementation of the models is performed using hardware consisting of an NVIDIA GeForce RTX 3050 GPU and the PyTorch framework. We first present and discuss results for keyword detection, then the results for signature and stamp detection.

A. Evaluation of selected OCR models

To ensure keyword detection reliability, we evaluate the two selected OCR methods: Microsoft Azure OCR and Tesseract. Since our dataset was not fully annotated for the OCR step, we propose to use these two OCR models in inference mode in order to produce annotations. Subsequently, we fine-tune the LayoutLMv3 model using these annotations and evaluate which of the two OCRs performs better with the fixed layout document analysis model. The training of LayoutLMv3 is performed under identical conditions, maintaining the same number of epochs and other hyperparameters.

As shown in Table V, Tesseract [12] demonstrates higher effectiveness with an F1-score of 0.81, indicating robust detection capabilities. In contrast, Microsoft Azure OCR's results are more conservative, with an F1-score of 0.56 and a precision

TABLE V: Comparison of fine-tuning LayoutLMv3 using Tesseract versus Microsoft Azure OCR.

Method	Validation Loss	F1-Score	Recall	Precision
Tesseract [12]	$1e^{-2}$	0.81	0.77	0.87
Azure OCR	$3.6e^{-2}$	0.56	0.48	0.68

of 0.68. However, the variation in the dataset size handled by each OCR tool is substantial, as depicted in Table VI. Azure OCR's ability to extract more instances on average provides a significant advantage, allowing the model to more accurately distinguish between irrelevant other (denoted as 'O') labels and targeted fields.

TABLE VI: Comparison of class instance counts.

Class	O	Title	Date	Client	Total	Total Value
Azure OCR	31715	217	242	553	315	221
Tesseract	6272	189	201	161	198	192

We present the inference results with annotations powered by Tesseract (cf. Figure 6) and Microsoft Azure OCR (cf. Figure 7). As discussed earlier following the results of Table VI, Azure OCR provides more selective and stable results, capturing target fields with minimal errors, whereas Tesseract exhibits less precision, often mislabeling or capturing non-target fields.

In conclusion, the robustness and selectivity of Microsoft Azure OCR make it the preferred choice for the critical task of text recognition.

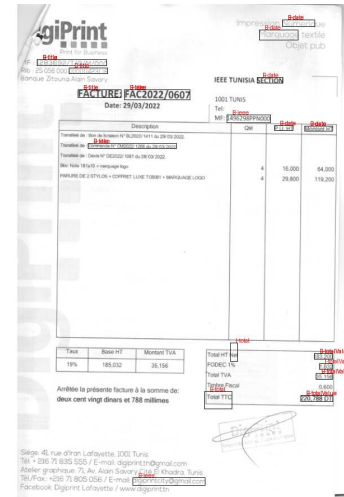


Fig. 6: LayoutLMv3 output using Tesseract.

B. Final keyword detection results

For document layout analysis, we train models on 50 epochs with a batch size of 4. We opt for Adam optimizer and L2 regularization with a weight decay of 10^{-3} . The learning rate is also fixed at $5e^{-5}$ for the three models.

Addressing data imbalance is a crucial step in this process. We opt for using the focal loss, which is an altered version of